

RFC-0005 - Modelo de Dados Detalhado (Schemas Segregados + DDLs Implementados)

Status: Aprovado e Implementado

Autor: Equipe de Arquitetura

Data: 07/05/2026

Depende de: RFC-0001, RFC-0002, RFC-0003, RFC-0004

1. Contexto e Evolução Arquitetural

Este RFC define:

- Modelo relacional estabelecido.
- Estrutura física das tabelas segregadas por domínios de negócio (Schemas).
- Índices obrigatórios.
- Estratégia de versionamento de schema.
- Preparação para evolução multi-tenant.
- Estratégia de migração futura (schema por tenant / banco por tenant).

Banco alvo: PostgreSQL.

1.1 A Segregação de Schemas

Diferente da proposta do MVP inicial que concentrava todos os domínios em um único schema `public`, a arquitetura evoluiu para a utilização de **schemas funcionais separados**. Essa decisão visa criar uma fachada de serviços limpa e garantir a governança dos dados. Foram estabelecidos dois schemas fundamentais:

- Schema pessoa:** Atua como o núcleo de identidade canônica global (independente do Keycloak), centralizando os dados demográficos, físicos, biológicos e estruturais de indivíduos, empresas e grupos governamentais.
- Schema folha_ponto:** Atua como o domínio específico multi-tenant, responsável por orquestrar organizações, unidades físicas, autorizações granulares e o registro de presença imutável.

2. Princípios de Modelagem (Diretrizes Arquiteturais)

- Auditoria:** Preparar colunas para auditoria desde o início: `criado_por`, `criado_em`, `atualizado_por` e `atualizado_em` operando em UTC. Adicionalmente, tabelas críticas implementam tabelas de histórico `_aud` ou triggers específicos no banco.

- **Biometria:** Deve ser referenciada de forma relacional na tabela de eventos, com os vetores (templates) armazenados em sistemas especializados de Storage (ex: SeaweedFS / S3) ou colunas adequadas para processamento.
 - **Defesa em Profundidade:** Constraints do banco de dados (CHECK, UNIQUE, ENUMS) são a última linha de defesa; nunca confiar apenas em validações de aplicação.
 - **Imutabilidade Real:** Tabelas de log e registro de eventos são estritamente imutáveis, servindo apenas para inclusão (append-only), garantindo a preservação do evento bruto conforme o princípio do Comando Único.
 - **Multi-tenancy:** Toda tabela de escopo organizacional no schema `folha_ponto` deve conter `organizacao_id` obrigatório para isolamento lógico. Tabelas do schema `pessoa` operam de forma global.
 - **Performance:** Índices B-Tree, espaciais (GIST) e únicos devem refletir os padrões de acesso reais estipulados no MVP.
-

3. Convenções (Texto Atualizado)

- **Chaves Primárias (Flexibilizadas):** A escolha do tipo de chave primária dependerá do contexto arquitetural da entidade:
 - **Inteiros Sequenciais (Padrão Majoritário):** A maioria das tabelas do sistema utilizará o tradicional número de tamanho grande (`int8` / `BIGINT GENERATED BY DEFAULT AS IDENTITY`), obrigatório e gerado automaticamente pelo banco caso não seja informado pelo usuário. Este formato garante alta densidade e performance ideal para os índices B-Tree (padrão já adotado globalmente no schema `pessoa`).
 - **UUIDs (Uso Restrito):** Entidades específicas de domínio isolado que necessitam de identificação global e ofuscação (como a `Organização` ou links externos) utilizarão UUID (gerados via `gen_random_uuid()`), evitando a exposição de sequências lógicas.
 - **Timezones:** `criado_em` e `atualizado_em` operam obrigatoriamente em UTC (timestamps sem e com timezone conforme o contexto).
 - **Timestamp Seguro:** `recebido_no_servidor_em` é obrigatório para registros críticos e representa a hora oficial do servidor, imune a fraudes no dispositivo cliente.
 - **Deleção:** Soft delete e status cycle onde necessário (ex: ENUM 'ativo', 'inativo', 'suspense'). Nunca deletar registros de presença contidos no evento bruto.
 - **Nomenclatura Explícita:** Todas as constraints (FKs, UQs, CHECKs) devem ser nomeadas explicitamente para facilitar a manutenção e o versionamento das tabelas.
-

4. O Schema: Pessoa

4.1 Introdução e Descrição Técnica

O schema `pessoa` é a fundação arquitetural da identidade global do sistema. Ele foi desenhado sob uma forte normalização e utiliza o padrão de **herança/subtipagem** de tabelas. No centro existe a

tabela raiz **pessoa**, da qual derivam ramificações altamente especializadas: **pessoa_fisica** (ancorada no CPF como chave canônica imutável da identidade), **pessoa_juridica** (CNPJ e atributos corporativos) e **pessoa_grupo** (para aglomerações e hierarquias).

Este schema é estritamente **global** (não possui conceito de **organizacao_id** ou **tenant**). Ele orquestra dados geográficos complexos (**local**, centróides e áreas via *PostGIS*), meios de comunicação (**contato**, segregados em email, telefone, endereço) e relacionamentos infinitos entre partes interessadas. As chaves primárias são inteiros pesados (**int8**) baseados em sequências (Sequences/Identity), o que otimiza profundamente as consultas de JOIN para dados centrais lidos frequentemente pelo sistema.

4.2 Definição Física do Banco (DDL)

```
-- DROP SCHEMA pessoa;
```

```
CREATE SCHEMA pessoa AUTHORIZATION backend;
```

```
-- DROP TYPE pessoa.classificacao_contato_enum;
```

```
CREATE TYPE pessoa.classificacao_contato_enum AS ENUM (  
    'residencial',  
    'comercial',  
    'outro');
```

```
-- DROP TYPE pessoa.estado_civil_enum;
```

```
CREATE TYPE pessoa.estado_civil_enum AS ENUM (  
    'solteiro',  
    'casado',  
    'divorciado',  
    'viuvo',  
    'uniao_estavel',  
    'separado',  
    'nao_informado');
```

```
-- DROP TYPE pessoa.nacionalidade_enum;
```

```
CREATE TYPE pessoa.nacionalidade_enum AS ENUM (  
    'brasileira',
```

```
    'estrangeira',  
    'nacionalizado',  
    'nao_informado');
```

```
-- DROP TYPE pessoa.papel_grupo_enum;
```

```
CREATE TYPE pessoa.papel_grupo_enum AS ENUM (  
    'gestor',  
    'membro',  
    'associado',  
    'interessado');
```

```
-- DROP TYPE pessoa.sexo_enum;
```

```
CREATE TYPE pessoa.sexo_enum AS ENUM (  
    'masculino',  
    'feminino',  
    'outro',  
    'nao_informado');
```

```
-- DROP TYPE pessoa.status_enum;
```

```
CREATE TYPE pessoa.status_enum AS ENUM (  
    'ativo',  
    'inativo',  
    'suspense');
```

```
-- DROP TYPE pessoa.tipo_contato_enum;
```

```
CREATE TYPE pessoa.tipo_contato_enum AS ENUM (  
    'telefone',  
    'email',  
    'endereco',  
    'pessoa_relacionada');
```

```

-- DROP TYPE pessoa.tipo_local_enum;

CREATE TYPE pessoa.tipo_local_enum AS ENUM (
    'pais',
    'estado',
    'municipio',
    'bairro',
    'logradouro',
    'regiao',
    'area_geografica',
    'estabelecimento',
    'local_generico');

-- DROP TYPE pessoa.tipo_pessoa_enum;

CREATE TYPE pessoa.tipo_pessoa_enum AS ENUM (
    'fisica',
    'juridica',
    'grupo');

-- DROP SEQUENCE pessoa.contato_id_seq;

CREATE SEQUENCE pessoa.contato_id_seq
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 9223372036854775807
    START 1
    CACHE 1
    NO CYCLE;

-- Permissions

ALTER SEQUENCE pessoa.contato_id_seq OWNER TO backend;
GRANT ALL ON SEQUENCE pessoa.contato_id_seq TO backend;

```

```
-- DROP SEQUENCE pessoa.local_id_seq;
```

```
CREATE SEQUENCE pessoa.local_id_seq  
    INCREMENT BY 1  
    MINVALUE 1  
    MAXVALUE 9223372036854775807  
    START 1  
    CACHE 1  
    NO CYCLE;
```

```
-- Permissions
```

```
ALTER SEQUENCE pessoa.local_id_seq OWNER TO backend;  
GRANT ALL ON SEQUENCE pessoa.local_id_seq TO backend;
```

```
-- DROP SEQUENCE pessoa.pessoa_contato_id_seq;
```

```
CREATE SEQUENCE pessoa.pessoa_contato_id_seq  
    INCREMENT BY 1  
    MINVALUE 1  
    MAXVALUE 9223372036854775807  
    START 1  
    CACHE 1  
    NO CYCLE;
```

```
-- Permissions
```

```
ALTER SEQUENCE pessoa.pessoa_contato_id_seq OWNER TO backend;  
GRANT ALL ON SEQUENCE pessoa.pessoa_contato_id_seq TO backend;
```

```
-- DROP SEQUENCE pessoa.pessoa_grupo_hierarquia_id_seq;
```

```
CREATE SEQUENCE pessoa.pessoa_grupo_hierarquia_id_seq  
    INCREMENT BY 1  
    MINVALUE 1
```

```
MAXVALUE 9223372036854775807
```

```
START 1
```

```
CACHE 1
```

```
NO CYCLE;
```

```
-- Permissions
```

```
ALTER SEQUENCE pessoa.pessoa_grupo_hierarquia_id_seq OWNER TO  
backend;
```

```
GRANT ALL ON SEQUENCE pessoa.pessoa_grupo_hierarquia_id_seq TO  
backend;
```

```
-- DROP SEQUENCE pessoa.pessoa_grupo_membros_id_seq;
```

```
CREATE SEQUENCE pessoa.pessoa_grupo_membros_id_seq
```

```
INCREMENT BY 1
```

```
MINVALUE 1
```

```
MAXVALUE 9223372036854775807
```

```
START 1
```

```
CACHE 1
```

```
NO CYCLE;
```

```
-- Permissions
```

```
ALTER SEQUENCE pessoa.pessoa_grupo_membros_id_seq OWNER TO  
backend;
```

```
GRANT ALL ON SEQUENCE pessoa.pessoa_grupo_membros_id_seq TO  
backend;
```

```
-- DROP SEQUENCE pessoa.pessoa_id_seq;
```

```
CREATE SEQUENCE pessoa.pessoa_id_seq
```

```
INCREMENT BY 1
```

```
MINVALUE 1
```

```
MAXVALUE 9223372036854775807
```

```
START 1
```

```
CACHE 1
NO CYCLE;

-- Permissions

ALTER SEQUENCE pessoa.pessoa_id_seq OWNER TO backend;
GRANT ALL ON SEQUENCE pessoa.pessoa_id_seq TO backend;

-- DROP SEQUENCE pessoa.pessoa_relacionamento_id_seq;

CREATE SEQUENCE pessoa.pessoa_relacionamento_id_seq
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 9223372036854775807
    START 1
    CACHE 1
    NO CYCLE;

-- Permissions

ALTER SEQUENCE pessoa.pessoa_relacionamento_id_seq OWNER TO
backend;
GRANT ALL ON SEQUENCE pessoa.pessoa_relacionamento_id_seq TO
backend;

-- DROP SEQUENCE pessoa.profissao_id_seq;

CREATE SEQUENCE pessoa.profissao_id_seq
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 9223372036854775807
    START 1
    CACHE 1
    NO CYCLE;

-- Permissions
```



```
ALTER SEQUENCE pessoa.profissao_id_seq OWNER TO backend;  
GRANT ALL ON SEQUENCE pessoa.profissao_id_seq TO backend;
```

```
-- DROP SEQUENCE pessoa.relacao_id_seq;
```

```
CREATE SEQUENCE pessoa.relacao_id_seq  
    INCREMENT BY 1  
    MINVALUE 1  
    MAXVALUE 9223372036854775807  
    START 1  
    CACHE 1  
    NO CYCLE;
```

```
-- Permissions
```

```
ALTER SEQUENCE pessoa.relacao_id_seq OWNER TO backend;  
GRANT ALL ON SEQUENCE pessoa.relacao_id_seq TO backend;
```

```
-- DROP SEQUENCE pessoa.relacao_tipo_funcao_id_seq;
```

```
CREATE SEQUENCE pessoa.relacao_tipo_funcao_id_seq  
    INCREMENT BY 1  
    MINVALUE 1  
    MAXVALUE 9223372036854775807  
    START 1  
    CACHE 1  
    NO CYCLE;
```

```
-- Permissions
```

```
ALTER SEQUENCE pessoa.relacao_tipo_funcao_id_seq OWNER TO backend;  
GRANT ALL ON SEQUENCE pessoa.relacao_tipo_funcao_id_seq TO  
backend;
```

```

-- DROP SEQUENCE pessoa.relacao_tipo_id_seq;

CREATE SEQUENCE pessoa.relacao_tipo_id_seq
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 9223372036854775807
    START 1
    CACHE 1
    NO CYCLE;

-- Permissions

ALTER SEQUENCE pessoa.relacao_tipo_id_seq OWNER TO backend;
GRANT ALL ON SEQUENCE pessoa.relacao_tipo_id_seq TO backend;
-- pessoa.contato definition

-- Drop table

-- DROP TABLE pessoa.contato;

CREATE TABLE pessoa.contato (
    id int8 GENERATED BY DEFAULT AS IDENTITY( INCREMENT BY 1
MINVALUE 1 MAXVALUE 9223372036854775807 START 1 CACHE 1 NO CYCLE)
NOT NULL,
    tipo pessoa.tipo_contato_enum NOT NULL,
    CONSTRAINT contato_pkey PRIMARY KEY (id)
);

-- Permissions

ALTER TABLE pessoa.contato OWNER TO backend;
GRANT ALL ON TABLE pessoa.contato TO backend;

-- pessoa.pessoa definition

```

-- Drop table

-- DROP TABLE pessoa.pessoa;

CREATE TABLE pessoa.pessoa (

 id int8 GENERATED BY DEFAULT AS IDENTITY(INCREMENT BY 1
MINVALUE 1 MAXVALUE 9223372036854775807 START 1 CACHE 1 NO CYCLE)
NOT NULL,

 nome text NOT NULL,

 tipo pessoa.tipo_pessoa_enum NOT NULL,

 status pessoa.status_enum DEFAULT 'ativo'::pessoa.status_enum
NOT NULL,

 observacao text NULL,

 data_inicio date NULL,

 data_termino date NULL,

 criado_por int8 NOT NULL,

 criado_em timestamptz DEFAULT now() NOT NULL,

 atualizado_por int8 NOT NULL,

 atualizado_em timestamptz DEFAULT now() NOT NULL,

 CONSTRAINT pessoa_pkey PRIMARY KEY (id)

);

CREATE INDEX idx_pessoa_status ON pessoa.pessoa USING btree
(status);

CREATE INDEX idx_pessoa_tipo ON pessoa.pessoa USING btree (tipo);

-- Table Triggers

create trigger trg_auditoria_pessoa before

insert

on

 pessoa.pessoa for each row execute function
pessoa.trg_set_auditoria();

-- Permissions

ALTER TABLE pessoa.pessoa OWNER TO backend;

GRANT ALL ON TABLE pessoa.pessoa TO backend;

```
-- pessoa.profissao definition

-- Drop table

-- DROP TABLE pessoa.profissao;

CREATE TABLE pessoa.profissao (
    id int8 GENERATED BY DEFAULT AS IDENTITY( INCREMENT BY 1
MINVALUE 1 MAXVALUE 9223372036854775807 START 1 CACHE 1 NO CYCLE)
NOT NULL,
    nome text NOT NULL,
    CONSTRAINT profissao_pkey PRIMARY KEY (id)
);

-- Permissions

ALTER TABLE pessoa.profissao OWNER TO backend;
GRANT ALL ON TABLE pessoa.profissao TO backend;

-- pessoa.relacao_tipo definition

-- Drop table

-- DROP TABLE pessoa.relacao_tipo;

CREATE TABLE pessoa.relacao_tipo (
    id int8 GENERATED BY DEFAULT AS IDENTITY( INCREMENT BY 1
MINVALUE 1 MAXVALUE 9223372036854775807 START 1 CACHE 1 NO CYCLE)
NOT NULL,
    nome text NOT NULL,
    CONSTRAINT relacao_tipo_pkey PRIMARY KEY (id)
);
```

```
CREATE UNIQUE INDEX uq_relacao_tipo_nome ON pessoa.relacao_tipo
USING btree (nome);
```

```
-- Permissions
```

```
ALTER TABLE pessoa.relacao_tipo OWNER TO backend;
GRANT ALL ON TABLE pessoa.relacao_tipo TO backend;
```

```
-- pessoa.contato_email definition
```

```
-- Drop table
```

```
-- DROP TABLE pessoa.contato_email;
```

```
CREATE TABLE pessoa.contato_email (
    id int8 NOT NULL,
    email text NOT NULL,
    email_normalizado text NOT NULL,
    CONSTRAINT contato_email_pkey PRIMARY KEY (id),
    CONSTRAINT contato_email_id_fkey FOREIGN KEY (id) REFERENCES
pessoa.contato(id)
);
```

```
CREATE UNIQUE INDEX uq_email_norm ON pessoa.contato_email USING
btree (email_normalizado);
```

```
-- Permissions
```

```
ALTER TABLE pessoa.contato_email OWNER TO backend;
GRANT ALL ON TABLE pessoa.contato_email TO backend;
```

```
-- pessoa.contato_pessoa_relacionada definition
```

```
-- Drop table
```

```

-- DROP TABLE pessoa.contato_pessoa_relacionada;

CREATE TABLE pessoa.contato_pessoa_relacionada (
    id int8 NOT NULL,
    id_pessoa_relacionada int8 NULL,
    nome text NULL,
    descricao text NULL,
    CONSTRAINT contato_pessoa_relacionada_pkey PRIMARY KEY (id),
    CONSTRAINT contato_pessoa_relacionada_id_fkey FOREIGN KEY (id)
REFERENCES pessoa.contato(id),

    CONSTRAINT
contato_pessoa_relacionada_id_pessoa_relacionada_fkey FOREIGN KEY
(id_pessoa_relacionada) REFERENCES pessoa.pessoa(id)
);

-- Permissions

ALTER TABLE pessoa.contato_pessoa_relacionada OWNER TO backend;
GRANT ALL ON TABLE pessoa.contato_pessoa_relacionada TO backend;

-- pessoa.contato_telefone definition

-- Drop table

-- DROP TABLE pessoa.contato_telefone;

CREATE TABLE pessoa.contato_telefone (
    id int8 NOT NULL,
    numero text NOT NULL,
    numero_normalizado text NOT NULL,
    CONSTRAINT contato_telefone_pkey PRIMARY KEY (id),
    CONSTRAINT contato_telefone_id_fkey FOREIGN KEY (id)
REFERENCES pessoa.contato(id)
);

CREATE UNIQUE INDEX uq_tel_norm ON pessoa.contato_telefone USING
btree (numero_normalizado);

```

```

-- Permissions

ALTER TABLE pessoa.contato_telefone OWNER TO backend;
GRANT ALL ON TABLE pessoa.contato_telefone TO backend;


-- pessoa."local" definition


-- Drop table


-- DROP TABLE pessoa."local";


CREATE TABLE pessoa."local" (
    id int8 GENERATED BY DEFAULT AS IDENTITY( INCREMENT BY 1
MINVALUE 1 MAXVALUE 9223372036854775807 START 1 CACHE 1 NO CYCLE)
NOT NULL,
    nome text NOT NULL,
    sigla text NULL,
    tipo pessoa.tipo_local_enum NOT NULL,
    id_local_pai int8 NULL,
    centroide public.geometry(point, 4326) NULL,
    area public.geometry(multipolygon, 4326) NULL,
    CONSTRAINT local_pkey PRIMARY KEY (id),
    CONSTRAINT local_id_local_pai_fkey FOREIGN KEY (id_local_pai)
REFERENCES pessoa."local"(id)
);

CREATE INDEX idx_local_area ON pessoa.local USING gist (area);
CREATE INDEX idx_local_pai ON pessoa.local USING btree
(id_local_pai);


-- Permissions

ALTER TABLE pessoa."local" OWNER TO backend;
GRANT ALL ON TABLE pessoa."local" TO backend;

```

```

-- pessoa.pessoa_contato definition

-- Drop table

-- DROP TABLE pessoa.pessoa_contato;

CREATE TABLE pessoa.pessoa_contato (
    id int8 GENERATED BY DEFAULT AS IDENTITY( INCREMENT BY 1
MINVALUE 1 MAXVALUE 9223372036854775807 START 1 CACHE 1 NO CYCLE)
NOT NULL,
    id_pessoa int8 NOT NULL,
    id_contato int8 NOT NULL,
    classificacao pessoa.classificacao_contato_enum NULL,
    prioridade int4 NULL,
    CONSTRAINT pessoa_contato_pkey PRIMARY KEY (id),
    CONSTRAINT pessoa_contato_id_contato_fkey FOREIGN KEY
(id_contato) REFERENCES pessoa.contato(id),
    CONSTRAINT pessoa_contato_id_pessoa_fkey FOREIGN KEY
(id_pessoa) REFERENCES pessoa.pessoa(id)
);

CREATE INDEX idx_pc_lookup ON pessoa.pessoa_contato USING btree
(id_pessoa, prioridade);

-- Permissions

ALTER TABLE pessoa.pessoa_contato OWNER TO backend;
GRANT ALL ON TABLE pessoa.pessoa_contato TO backend;

-- pessoa.pessoa_fisica definition

-- Drop table

-- DROP TABLE pessoa.pessoa_fisica;

```



```

CREATE TABLE pessoa.pessoa_fisica (
    id int8 NOT NULL,
    cpf varchar(11) NULL,
    rg text NULL,
    apelido text NULL,
    id_pessoa_mae int8 NULL,
    id_pessoa_pai int8 NULL,
    nome_mae text NULL,
    nome_pai text NULL,
    local_nascimento_nome text NULL,
    id_local_nascimento int8 NULL,
    local_falecimento_nome text NULL,
    id_local_falecimento int8 NULL,
    sexo pessoa.sexo_enum NULL,
    estado_civil pessoa.estado_civil_enum NULL,
    nacionalidade pessoa.nacionalidade_enum NULL,
    id_profissao int8 NULL,
    profissao_nome text NULL,
    CONSTRAINT pessoa_fisica_pkey PRIMARY KEY (id),
    CONSTRAINT pessoa_fisica_id_fkey FOREIGN KEY (id) REFERENCES
pessoa.pessoa(id) ON DELETE CASCADE,
    CONSTRAINT pessoa_fisica_id_local_falecimento_fkey FOREIGN KEY
(id_local_falecimento) REFERENCES pessoa."local"(id),
    CONSTRAINT pessoa_fisica_id_local_nascimento_fkey FOREIGN KEY
(id_local_nascimento) REFERENCES pessoa."local"(id),
    CONSTRAINT pessoa_fisica_id_pessoa_mae_fkey FOREIGN KEY
(id_pessoa_mae) REFERENCES pessoa.pessoa(id),
    CONSTRAINT pessoa_fisica_id_pessoa_pai_fkey FOREIGN KEY
(id_pessoa_pai) REFERENCES pessoa.pessoa(id),
    CONSTRAINT pessoa_fisica_id_profissao_fkey FOREIGN KEY
(id_profissao) REFERENCES pessoa.profissao(id)
);

CREATE UNIQUE INDEX uq_pf_cpf ON pessoa.pessoa_fisica USING btree
(cpf) WHERE (cpf IS NOT NULL);

-- Table Triggers

```

```
create trigger trg_pf_inativo after
insert
    or
update
    on
    pessoa.pessoa_fisica for each row execute function
pessoa.trg_inativar_pf();
create trigger trg_validar_pais before
insert
    or
update
    on
    pessoa.pessoa_fisica for each row execute function
pessoa.trg_validar_pais();

-- Permissions

ALTER TABLE pessoa.pessoa_fisica OWNER TO backend;
GRANT ALL ON TABLE pessoa.pessoa_fisica TO backend;

-- pessoa.pessoa_grupo definition

-- Drop table

-- DROP TABLE pessoa.pessoa_grupo;

CREATE TABLE pessoa.pessoa_grupo (
    id int8 NOT NULL,
    descricao text NULL,
    nome_resumido_sigla text NULL,
    CONSTRAINT pessoa_grupo_pkey PRIMARY KEY (id),
    CONSTRAINT pessoa_grupo_id_fkey FOREIGN KEY (id) REFERENCES
pessoa.pessoa(id) ON DELETE CASCADE
);
```

-- Table Triggers

```
create trigger trg_grupo_inativo after
insert
    or
update
    on
    pessoa.pessoa_grupo for each row execute function
pessoa.trg_inativar_grupo();
```

-- Permissions

```
ALTER TABLE pessoa.pessoa_grupo OWNER TO backend;
GRANT ALL ON TABLE pessoa.pessoa_grupo TO backend;
```

-- pessoa.pessoa_grupo_hierarquia definition

-- Drop table

-- DROP TABLE pessoa.pessoa_grupo_hierarquia;

```
CREATE TABLE pessoa.pessoa_grupo_hierarquia (
    id int8 GENERATED BY DEFAULT AS IDENTITY( INCREMENT BY 1
MINVALUE 1 MAXVALUE 9223372036854775807 START 1 CACHE 1 NO CYCLE)
NOT NULL,
    id_grupo_pai int8 NOT NULL,
    id_grupo_filho int8 NOT NULL,
    nome_pai_para_filho varchar(255) NULL,
    nome_filho_para_pai varchar(255) NULL,
    CONSTRAINT pessoa_grupo_hierarquia_pkey PRIMARY KEY (id),
    CONSTRAINT pessoa_grupo_hierarquia_id_grupo_filho_fkey FOREIGN
KEY (id_grupo_filho) REFERENCES pessoa.pessoa_grupo(id),
    CONSTRAINT pessoa_grupo_hierarquia_id_grupo_pai_fkey FOREIGN
KEY (id_grupo_pai) REFERENCES pessoa.pessoa_grupo(id)
);
```

```
-- Permissions
```

```
ALTER TABLE pessoa.pessoa_grupo_hierarquia OWNER TO backend;  
GRANT ALL ON TABLE pessoa.pessoa_grupo_hierarquia TO backend;
```

```
-- pessoa.pessoa_grupo_membros definition
```

```
-- Drop table
```

```
-- DROP TABLE pessoa.pessoa_grupo_membros;
```

```
CREATE TABLE pessoa.pessoa_grupo_membros (  
    id int8 GENERATED BY DEFAULT AS IDENTITY( INCREMENT BY 1  
MINVALUE 1 MAXVALUE 9223372036854775807 START 1 CACHE 1 NO CYCLE)  
NOT NULL,  
    id_grupo int8 NOT NULL,  
    id_pessoa int8 NOT NULL,  
    papel pessoa.papel_grupo_enum NOT NULL,  
    descricao_funcao varchar(255) NULL,  
    data_entrada timestamptz DEFAULT now() NULL,  
    data_saida timestamptz NULL,  
    CONSTRAINT pessoa_grupo_membros_pkey PRIMARY KEY (id),  
    CONSTRAINT pessoa_grupo_membros_id_grupo_fkey FOREIGN KEY  
(id_grupo) REFERENCES pessoa.pessoa_grupo(id),  
    CONSTRAINT pessoa_grupo_membros_id_pessoa_fkey FOREIGN KEY  
(id_pessoa) REFERENCES pessoa.pessoa(id)  
);  
  
CREATE UNIQUE INDEX uq_pg_membro_ativo ON  
pessoa.pessoa_grupo_membros USING btree (id_pessoa, id_grupo,  
papel) WHERE (data_saida IS NULL);
```

```
-- Permissions
```

```
ALTER TABLE pessoa.pessoa_grupo_membros OWNER TO backend;  
GRANT ALL ON TABLE pessoa.pessoa_grupo_membros TO backend;
```

```

-- pessoa.pessoa_juridica definition

-- Drop table

-- DROP TABLE pessoa.pessoa_juridica;

CREATE TABLE pessoa.pessoa_juridica (
    id int8 NOT NULL,
    cnpj varchar(14) NULL,
    inscricao_estadual text NULL,
    inscricao_municipal text NULL,
    razao_social text NOT NULL,
    sigla text NULL,
    local_sede_nome text NULL,
    id_local_sede int8 NULL,
    natureza_juridica text NULL,
    regime_tributario text NULL,
    porte text NULL,
    capital_social numeric(15, 2) NULL,
    id_empresa_controladora int8 NULL,
    CONSTRAINT pessoa_juridica_pkey PRIMARY KEY (id),
    CONSTRAINT pessoa_juridica_id_empresa_controladora_fkey
FOREIGN KEY (id_empresa_controladora) REFERENCES
pessoa.pessoa_juridica(id),
    CONSTRAINT pessoa_juridica_id_fkey FOREIGN KEY (id) REFERENCES
pessoa.pessoa(id) ON DELETE CASCADE,
    CONSTRAINT pessoa_juridica_id_local_sede_fkey FOREIGN KEY
(id_local_sede) REFERENCES pessoa."local"(id)
);

CREATE INDEX idx_pj_inscricao_estadual ON pessoa.pessoa_juridica
USING btree (inscricao_estadual);

CREATE INDEX idx_pj_inscricao_municipal ON pessoa.pessoa_juridica
USING btree (inscricao_municipal);

CREATE UNIQUE INDEX uq_pj_cnpj ON pessoa.pessoa_juridica USING
btree (cnpj) WHERE (cnpj IS NOT NULL);

```

-- Table Triggers

```
create trigger trg_pj_inativo after
insert
    or
update
    on
    pessoa.pessoa_juridica for each row execute function
pessoa.trg_inativar_pj();
```

-- Permissions

```
ALTER TABLE pessoa.pessoa_juridica OWNER TO backend;
GRANT ALL ON TABLE pessoa.pessoa_juridica TO backend;
```

-- pessoa.relacao definition

-- Drop table

-- DROP TABLE pessoa.relacao;

```
CREATE TABLE pessoa.relacao (
    id int8 GENERATED BY DEFAULT AS IDENTITY( INCREMENT BY 1
MINVALUE 1 MAXVALUE 9223372036854775807 START 1 CACHE 1 NO CYCLE)
NOT NULL,
    id_relacao_tipo int8 NOT NULL,
    CONSTRAINT relacao_pkey PRIMARY KEY (id),
    CONSTRAINT relacao_id_relacao_tipo_fkey FOREIGN KEY
(id_relacao_tipo) REFERENCES pessoa.relacao_tipo(id)
);
```

-- Permissions

```
ALTER TABLE pessoa.relacao OWNER TO backend;
GRANT ALL ON TABLE pessoa.relacao TO backend;
```

```

-- pessoa.relacao_tipo_funcao definition

-- Drop table

-- DROP TABLE pessoa.relacao_tipo_funcao;

CREATE TABLE pessoa.relacao_tipo_funcao (
    id int8 GENERATED BY DEFAULT AS IDENTITY( INCREMENT BY 1
MINVALUE 1 MAXVALUE 9223372036854775807 START 1 CACHE 1 NO CYCLE)
NOT NULL,
    id_relacao_tipo int8 NOT NULL,
    nome text NOT NULL,
    sexo pessoa.sexo_enum NULL,
    CONSTRAINT relacao_tipo_funcao_pkey PRIMARY KEY (id),
    CONSTRAINT relacao_tipo_funcao_id_relacao_tipo_fkey FOREIGN
KEY (id_relacao_tipo) REFERENCES pessoa.relacao_tipo(id)
);

CREATE UNIQUE INDEX uq_relacao_tipo_funcao ON
pessoa.relacao_tipo_funcao USING btree (id_relacao_tipo, nome);

-- Permissions

ALTER TABLE pessoa.relacao_tipo_funcao OWNER TO backend;
GRANT ALL ON TABLE pessoa.relacao_tipo_funcao TO backend;

-- pessoa.contato_endereco definition

-- Drop table

-- DROP TABLE pessoa.contato_endereco;

CREATE TABLE pessoa.contato_endereco (
    id int8 NOT NULL,

```

```

    cep varchar(10) NULL,
    logradouro text NULL,
    numero text NULL,
    complemento text NULL,
    bairro text NULL,
    referencia text NULL,
    id_local int8 NULL,
    localizacao public.geometry(point, 4326) NULL,
    area public.geometry(multipolygon, 4326) NULL,
    hash_endereco bytea NULL,
    CONSTRAINT contato_endereco_pkey PRIMARY KEY (id),
    CONSTRAINT contato_endereco_id_fkey FOREIGN KEY (id)
REFERENCES pessoa.contato(id),
    CONSTRAINT contato_endereco_id_local_fkey FOREIGN KEY
(id_local) REFERENCES pessoa."local"(id)
);
CREATE UNIQUE INDEX uq_endereco_hash ON pessoa.contato_endereco
USING btree (hash_endereco);

```

```
-- Table Triggers
```

```

create trigger trg_hash_endereco before
insert
    or
update
    on
    pessoa.contato_endereco for each row execute function
pessoa.trg_hash_endereco();

```

```
-- Permissions
```

```

ALTER TABLE pessoa.contato_endereco OWNER TO backend;
GRANT ALL ON TABLE pessoa.contato_endereco TO backend;

```

```
-- pessoa.pessoa_relacionamento definition
```



```

-- Drop table

-- DROP TABLE pessoa.pessoa_relacionamento;

CREATE TABLE pessoa.pessoa_relacionamento (
    id int8 GENERATED BY DEFAULT AS IDENTITY( INCREMENT BY 1
MINVALUE 1 MAXVALUE 9223372036854775807 START 1 CACHE 1 NO CYCLE)
NOT NULL,
    id_relacao int8 NOT NULL,
    id_pessoa int8 NOT NULL,
    id_relacao_tipo_funcao int8 NOT NULL,
    data_inicio timestamptz DEFAULT now() NULL,
    data_fim timestamptz NULL,
    CONSTRAINT pessoa_relacionamento_pkey PRIMARY KEY (id),
    CONSTRAINT pessoa_relacionamento_id_pessoa_fkey FOREIGN KEY
(id_pessoa) REFERENCES pessoa.pessoa(id),
    CONSTRAINT pessoa_relacionamento_id_relacao_fkey FOREIGN KEY
(id_relacao) REFERENCES pessoa.relacao(id),
    CONSTRAINT pessoa_relacionamento_id_relacao_tipo_funcao_fkey
FOREIGN KEY (id_relacao_tipo_funcao) REFERENCES
pessoa.relacao_tipo_funcao(id)
);

CREATE INDEX idx_pr_pessoa ON pessoa.pessoa_relacionamento USING
btree (id_pessoa);

CREATE INDEX idx_pr_relacao ON pessoa.pessoa_relacionamento USING
btree (id_relacao);

CREATE UNIQUE INDEX uq_relacao_pessoa_funcao_ativa ON
pessoa.pessoa_relacionamento USING btree (id_relacao, id_pessoa,
id_relacao_tipo_funcao) WHERE (data_fim IS NULL);

-- Permissions

ALTER TABLE pessoa.pessoa_relacionamento OWNER TO backend;
GRANT ALL ON TABLE pessoa.pessoa_relacionamento TO backend;

-- pessoa.vw_grupo_caminho source

```

```

CREATE OR REPLACE VIEW pessoa.vw_grupo_caminho
AS WITH RECURSIVE arvore AS (
    SELECT h.id_grupo_pai,
           h.id_grupo_filho,
           (p1.nome || ' > '::text) || p2.nome AS caminho
    FROM pessoa.pessoa_grupo_hierarquia h
         JOIN pessoa.pessoa p1 ON p1.id = h.id_grupo_pai
         JOIN pessoa.pessoa p2 ON p2.id = h.id_grupo_filho
    UNION ALL
    SELECT a.id_grupo_pai,
           h.id_grupo_filho,
           (a.caminho || ' > '::text) || p.nome
    FROM pessoa.pessoa_grupo_hierarquia h
         JOIN arvore a ON a.id_grupo_filho = h.id_grupo_pai
         JOIN pessoa.pessoa p ON p.id = h.id_grupo_filho
)
SELECT id_grupo_pai,
       id_grupo_filho,
       caminho
FROM arvore;

```

```
-- Permissions
```

```

ALTER TABLE pessoa.vw_grupo_caminho OWNER TO backend;
GRANT ALL ON TABLE pessoa.vw_grupo_caminho TO backend;

```

```
-- pessoa.vw_grupo_hierarquia_asc source
```

```

CREATE OR REPLACE VIEW pessoa.vw_grupo_hierarquia_asc
AS WITH RECURSIVE arvore AS (
    SELECT h.id_grupo_pai,
           h.id_grupo_filho,
           1 AS nivel

```

```

        FROM pessoa.pessoa_grupo_hierarquia h
    UNION ALL
        SELECT h.id_grupo_pai,
               a.id_grupo_filho,
               a.nivel + 1
        FROM pessoa.pessoa_grupo_hierarquia h
             JOIN arvore a ON h.id_grupo_filho = a.id_grupo_pai
    )
    SELECT id_grupo_pai,
           id_grupo_filho,
           nivel
    FROM arvore;

-- Permissions

ALTER TABLE pessoa.vw_grupo_hierarquia_asc OWNER TO backend;
GRANT ALL ON TABLE pessoa.vw_grupo_hierarquia_asc TO backend;

-- pessoa.vw_grupo_hierarquia_desc source

CREATE OR REPLACE VIEW pessoa.vw_grupo_hierarquia_desc
AS WITH RECURSIVE arvore AS (
    SELECT h.id_grupo_pai,
           h.id_grupo_filho,
           1 AS nivel,
           ARRAY[h.id_grupo_pai, h.id_grupo_filho] AS caminho
    FROM pessoa.pessoa_grupo_hierarquia h
    UNION ALL
    SELECT a.id_grupo_pai,
           h.id_grupo_filho,
           a.nivel + 1,
           a.caminho || h.id_grupo_filho
    FROM pessoa.pessoa_grupo_hierarquia h
         JOIN arvore a ON a.id_grupo_filho = h.id_grupo_pai

```

```
        )
SELECT id_grupo_pai,
       id_grupo_filho,
       nivel,
       caminho
FROM arvore;
```

```
-- Permissions
```

```
ALTER TABLE pessoa.vw_grupo_hierarquia_desc OWNER TO backend;
GRANT ALL ON TABLE pessoa.vw_grupo_hierarquia_desc TO backend;
```

```
-- pessoa.vw_grupo_hierarquia_nomeada source
```

```
CREATE OR REPLACE VIEW pessoa.vw_grupo_hierarquia_nomeada
AS SELECT h.id_grupo_pai,
         p1.nome AS nome_grupo_pai,
         h.id_grupo_filho,
         p2.nome AS nome_grupo_filho,
         h.nome_pai_para_filho,
         h.nome_filho_para_pai
FROM pessoa.pessoa_grupo_hierarquia h
     JOIN pessoa.pessoa p1 ON p1.id = h.id_grupo_pai
     JOIN pessoa.pessoa p2 ON p2.id = h.id_grupo_filho;
```

```
-- Permissions
```

```
ALTER TABLE pessoa.vw_grupo_hierarquia_nomeada OWNER TO backend;
GRANT ALL ON TABLE pessoa.vw_grupo_hierarquia_nomeada TO backend;
```

```
-- pessoa.vw_grupo_membros_ativos source
```

```
CREATE OR REPLACE VIEW pessoa.vw_grupo_membros_ativos
```

```

AS SELECT id,
        id_grupo,
        id_pessoa,
        papel,
        descricao_funcao,
        data_entrada,
        data_saida
FROM pessoa.pessoa_grupo_membros
WHERE data_saida IS NULL;

-- Permissions

ALTER TABLE pessoa.vw_grupo_membros_ativos OWNER TO backend;
GRANT ALL ON TABLE pessoa.vw_grupo_membros_ativos TO backend;

-- pessoa.vw_grupo_membros_hierarquia source

CREATE OR REPLACE VIEW pessoa.vw_grupo_membros_hierarquia
AS WITH RECURSIVE arvore AS (
        SELECT g.id AS id_grupo_raiz,
               g.id AS id_grupo
        FROM pessoa.pessoa_grupo g
    UNION ALL
        SELECT a_1.id_grupo_raiz,
               h.id_grupo_filho
        FROM pessoa.pessoa_grupo_hierarquia h
             JOIN arvore a_1 ON a_1.id_grupo = h.id_grupo_pai
    )
SELECT a.id_grupo_raiz AS id_grupo,
       gm.id_pessoa,
       gm.papel,
       gm.descricao_funcao
FROM arvore a

```

```

        JOIN pessoa.pessoa_grupo_membros gm ON gm.id_grupo =
a.id_grupo
    WHERE gm.data_saida IS NULL;

-- Permissions

ALTER TABLE pessoa.vw_grupo_membros_hierarquia OWNER TO backend;
GRANT ALL ON TABLE pessoa.vw_grupo_membros_hierarquia TO backend;


-- DROP FUNCTION pessoa.normalizar_texto(text);

CREATE OR REPLACE FUNCTION pessoa.normalizar_texto(t text)
    RETURNS text
    LANGUAGE plpgsql
    IMMUTABLE
AS $function$
BEGIN
    RETURN lower(trim(coalesce(t, '')));
END;
$function$
;

-- Permissions

ALTER FUNCTION pessoa.normalizar_texto(text) OWNER TO backend;
GRANT ALL ON FUNCTION pessoa.normalizar_texto(text) TO backend;


-- DROP FUNCTION pessoa.trg_hash_endereco();

CREATE OR REPLACE FUNCTION pessoa.trg_hash_endereco()
    RETURNS trigger
    LANGUAGE plpgsql
AS $function$

```

```

BEGIN
    NEW.hash_endereco :=
        digest(
            normalizar_texto(NEW.cep) ||
            normalizar_texto(NEW.logradouro) ||
            normalizar_texto(NEW.numero) ||
            normalizar_texto(NEW.complemento) ||
            normalizar_texto(NEW.bairro) ||
            coalesce(NEW.id_local::text, ''),
            'sha256'
        );
    RETURN NEW;
END;
$function$
;

-- Permissions

ALTER FUNCTION pessoa.trg_hash_endereco() OWNER TO backend;
GRANT ALL ON FUNCTION pessoa.trg_hash_endereco() TO backend;

-- DROP FUNCTION pessoa.trg_inativar_grupo();

CREATE OR REPLACE FUNCTION pessoa.trg_inativar_grupo()
    RETURNS trigger
    LANGUAGE plpgsql
AS $function$
BEGIN
    IF NEW.data_termino IS NOT NULL THEN
        UPDATE pessoa SET status = 'inativo' WHERE id = NEW.id;
    END IF;
    RETURN NEW;
END;
$function$
;

```

```
-- Permissions
```

```
ALTER FUNCTION pessoa.trg_inativar_grupo() OWNER TO backend;  
GRANT ALL ON FUNCTION pessoa.trg_inativar_grupo() TO backend;
```

```
-- DROP FUNCTION pessoa.trg_inativar_pf();
```

```
CREATE OR REPLACE FUNCTION pessoa.trg_inativar_pf()  
  RETURNS trigger  
  LANGUAGE plpgsql  
AS $function$  
BEGIN  
    IF NEW.local_falecimento_nome IS NOT NULL  
        OR NEW.id_local_falecimento IS NOT NULL THEN  
        UPDATE pessoa SET status = 'inativo' WHERE id = NEW.id;  
    END IF;  
    RETURN NEW;  
END;  
$function$  
;
```

```
-- Permissions
```

```
ALTER FUNCTION pessoa.trg_inativar_pf() OWNER TO backend;  
GRANT ALL ON FUNCTION pessoa.trg_inativar_pf() TO backend;
```

```
-- DROP FUNCTION pessoa.trg_inativar_pj();
```

```
CREATE OR REPLACE FUNCTION pessoa.trg_inativar_pj()  
  RETURNS trigger  
  LANGUAGE plpgsql  
AS $function$  
BEGIN
```



```

        IF NEW.local_sede_nome IS NULL AND NEW.id_local_sede IS NULL
THEN
            RETURN NEW;
        END IF;
        IF EXISTS (
            SELECT 1 FROM pessoa p WHERE p.id = NEW.id AND
p.data_termينو IS NOT NULL
        ) THEN
            UPDATE pessoa SET status = 'inativo' WHERE id = NEW.id;
        END IF;
        RETURN NEW;
END;
$function$
;

-- Permissions

ALTER FUNCTION pessoa.trg_inativar_pj() OWNER TO backend;
GRANT ALL ON FUNCTION pessoa.trg_inativar_pj() TO backend;

-- DROP FUNCTION pessoa.trg_set_auditoria();

CREATE OR REPLACE FUNCTION pessoa.trg_set_auditoria()
    RETURNS trigger
    LANGUAGE plpgsql
AS $function$
BEGIN
    IF TG_OP = 'INSERT' THEN
        NEW.atualizado_por := NEW.criado_por;
        NEW.atualizado_em := NEW.criado_em;
    END IF;
    RETURN NEW;
END;
$function$
;

```

```
-- Permissions
```

```
ALTER FUNCTION pessoa.trg_set_auditoria() OWNER TO backend;  
GRANT ALL ON FUNCTION pessoa.trg_set_auditoria() TO backend;
```

```
-- DROP FUNCTION pessoa.trg_validar_pais();
```

```
CREATE OR REPLACE FUNCTION pessoa.trg_validar_pais()  
    RETURNS trigger  
    LANGUAGE plpgsql  
AS $function$  
DECLARE sexo_mae sexo_enum;  
DECLARE sexo_pai sexo_enum;  
BEGIN  
    IF NEW.id_pessoa_mae IS NOT NULL THEN  
        SELECT sexo INTO sexo_mae FROM pessoa_fisica WHERE id =  
NEW.id_pessoa_mae;  
        IF sexo_mae <> 'feminino' THEN  
            RAISE EXCEPTION 'Mãe inválida';  
        END IF;  
    END IF;  
  
    IF NEW.id_pessoa_pai IS NOT NULL THEN  
        SELECT sexo INTO sexo_pai FROM pessoa_fisica WHERE id =  
NEW.id_pessoa_pai;  
        IF sexo_pai <> 'masculino' THEN  
            RAISE EXCEPTION 'Pai inválido';  
        END IF;  
    END IF;  
  
    RETURN NEW;  
END;  
$function$  
;
```

```
-- Permissions
```

```
ALTER FUNCTION pessoa.trg_validar_pais() OWNER TO backend;  
GRANT ALL ON FUNCTION pessoa.trg_validar_pais() TO backend;
```

```
-- Permissions
```

```
GRANT ALL ON SCHEMA pessoa TO backend;
```

4.3 Resumo Técnico e Diretrizes do Schema

- **Subtipagem e Chaves Canônicas:** A tabela raiz `pessoa` detém o status e a auditoria básica. As tabelas `pessoa_fisica` e `pessoa_juridica` usam a mesma PK (chave estrangeira para a raiz) e definem índices únicos fortes para `cpf` e `cnpj`, barrando absolutamente a duplicidade na camada de banco.
 - **Tipagem Restrita:** O schema faz uso intensivo de Custom Types (ENUMs do PostgreSQL) como `sexo_enum`, `tipo_pessoa_enum`, `status_enum`, injetando restrições de validação nativas e rápidas.
 - **Auditoria Baseada em Triggers:** Em vez de depender do Java para popular os dados de trilha e de status inativo em todas as ramificações, o schema utiliza funções nativas do PL/pgSQL (como `trg_set_auditoria` e `trg_inativar_pf`) engatilhadas (Triggers) em operações de `INSERT` ou `UPDATE`, blindando a integridade independente da camada de software.
 - **Normalização Geográfica e de Contato:** Contatos (telefone, email, endereço) e locais são desvinculados diretamente da pessoa. Eles nascem em tabelas mestre (ex: `contato_endereco` usando função de `hash_endereco` para deduplicação) e são pivotados pela tabela associativa `pessoa_contato`, permitindo que múltiplas pessoas compartilhem o mesmo endereço físico sem repetição de strings.
 - **Hierarquia e Views (ReBAC Foundation):** Grupos suportam relações estruturadas com a tabela `pessoa_grupo_hierarquia`. Para facilitar a vida dos desenvolvedores na extração de caminhos e níveis hierárquicos, o schema expõe diversas *Views Recursivas* (como `vw_grupo_caminho`, `vw_grupo_hierarquia_asc`, e `vw_grupo_membros_ativos`), transformando grafos complexos em consultas planas simples de serem consumidas.
-

5. O Schema: Folha_Ponto

5.1 Introdução e Descrição Técnica

O schema `folha_ponto` concentra as fronteiras lógicas do negócio focado na gestão temporal e controle de eventos. Diferente do schema de pessoas, este módulo é **100% desenhado para o modelo multi-tenant** (isolamento de software). Salvo entidades puramente sistêmicas de log e

perfis globais, a esmagadora maioria das tabelas possui o pilar `organizacao_id`, que divide as águas entre diferentes órgãos, secretarias ou clientes corporativos que consumirão o SaaS.

Ele utiliza identificadores Universais (UUID), gerados por `gen_random_uuid()`, o que facilita a geração de chaves seguras diretamente pela aplicação ou pelo banco. Ele consolida a infraestrutura para a **Arquitetura Antifraude** através de cadastros de `dispositivo`, armazenamentos biométricos (`perfil_biometrico`) e uma tabela central `registro_presenca`, que aceita coordenadas geográficas soltas (para confrontar com o polígono nativo da unidade).

5.2 Definição Física do Banco (DDL)

```
-- DROP SCHEMA folha_ponto;
```



```
CREATE SCHEMA folha_ponto AUTHORIZATION backend;
```



```
-- folha_ponto.dispositivo definition
```



```
-- Drop table
```



```
-- DROP TABLE folha_ponto.dispositivo;
```



```
CREATE TABLE folha_ponto.dispositivo (  
    id uuid DEFAULT gen_random_uuid() NOT NULL,  
    identificador_dispositivo varchar(128) NOT NULL,  
    usuario_id varchar(64) NULL,  
    nivel_confianca varchar(16) NOT NULL,  
    plataforma varchar(32) NULL,  
    criado_por varchar(64) NOT NULL,  
    criado_em timestamp NOT NULL,  
    atualizado_por varchar(64) NULL,  
    atualizado_em timestamp NULL,  
    CONSTRAINT dispositivo_identificador_dispositivo_key UNIQUE  
(identificador_dispositivo),  
    CONSTRAINT dispositivo_pkey PRIMARY KEY (id)  
);
```



```
-- Permissions
```

```
ALTER TABLE folha_ponto.dispositivo OWNER TO backend;
GRANT ALL ON TABLE folha_ponto.dispositivo TO backend;

-- folha_ponto.log_auditoria definition

-- Drop table

-- DROP TABLE folha_ponto.log_auditoria;

CREATE TABLE folha_ponto.log_auditoria (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    organizacao_id uuid NULL,
    acao varchar(128) NOT NULL,
    tipo_entidade varchar(64) NULL,
    entidade_id uuid NULL,
    decisao varchar(16) NULL,
    metadados jsonb NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,
    CONSTRAINT log_auditoria_pkey PRIMARY KEY (id)
);

CREATE INDEX idx_auditoria_org_tempo ON folha_ponto.log_auditoria
USING btree (organizacao_id, criado_em);

-- Permissions

ALTER TABLE folha_ponto.log_auditoria OWNER TO backend;
GRANT ALL ON TABLE folha_ponto.log_auditoria TO backend;
```

```
-- folha_ponto.organizacao definition

-- Drop table

-- DROP TABLE folha_ponto.organizacao;

CREATE TABLE folha_ponto.organizacao (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    nome varchar(255) NOT NULL,
    status varchar(32) DEFAULT 'ATIVO'::character varying NOT
NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,
    atualizado_por varchar(64) NULL,
    atualizado_em timestamp NULL,
    CONSTRAINT organizacao_pkey PRIMARY KEY (id)
);

CREATE INDEX idx_org_criado_por ON folha_ponto.organizacao USING
btree (criado_por);

-- Permissions

ALTER TABLE folha_ponto.organizacao OWNER TO backend;
GRANT ALL ON TABLE folha_ponto.organizacao TO backend;

-- folha_ponto.perfil_biometrico definition

-- Drop table

-- DROP TABLE folha_ponto.perfil_biometrico;

CREATE TABLE folha_ponto.perfil_biometrico (
```

```

        id uuid DEFAULT gen_random_uuid() NOT NULL,
        usuario_id varchar(64) NOT NULL,
        modelo_biometrico bytea NOT NULL,
        versao_modelo varchar(32) NOT NULL,
        criado_por varchar(64) NOT NULL,
        criado_em timestamp NOT NULL,
        atualizado_por varchar(64) NULL,
        atualizado_em timestamp NULL,
        CONSTRAINT perfil_biometrico_pkey PRIMARY KEY (id)
    );

-- Permissions

ALTER TABLE folha_ponto.perfil_biometrico OWNER TO backend;
GRANT ALL ON TABLE folha_ponto.perfil_biometrico TO backend;

-- folha_ponto.consentimento definition

-- Drop table

-- DROP TABLE folha_ponto.consentimento;

CREATE TABLE folha_ponto.consentimento (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    organizacao_id uuid NOT NULL,
    usuario_id varchar(64) NOT NULL,
    tipo_consentimento varchar(64) NOT NULL,
    status varchar(32) DEFAULT 'ATIVO'::character varying NOT
NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,

```

```
    atualizado_por varchar(64) NULL,  
    atualizado_em timestamp NULL,  
    CONSTRAINT consentimento_pkey PRIMARY KEY (id),  
    CONSTRAINT consentimento_organizacao_id_fkey FOREIGN KEY  
(organizacao_id) REFERENCES folha_ponto.organizacao(id)  
);
```

```
-- Permissions
```

```
ALTER TABLE folha_ponto.consentimento OWNER TO backend;  
GRANT ALL ON TABLE folha_ponto.consentimento TO backend;
```

```
-- folha_ponto.excecao_permissao_usuario definition
```

```
-- Drop table
```

```
-- DROP TABLE folha_ponto.excecao_permissao_usuario;
```

```
CREATE TABLE folha_ponto.excecao_permissao_usuario (  
    id uuid DEFAULT gen_random_uuid() NOT NULL,  
    organizacao_id uuid NOT NULL,  
    usuario_id varchar(64) NOT NULL,  
    recurso varchar(128) NOT NULL,  
    escopo varchar(64) NOT NULL,  
    condicao_json jsonb NULL,  
    criado_por varchar(64) NOT NULL,  
    criado_em timestamp NOT NULL,  
    atualizado_por varchar(64) NULL,  
    atualizado_em timestamp NULL,  
    CONSTRAINT excecao_permissao_usuario_pkey PRIMARY KEY (id),
```



```
        CONSTRAINT excecao_permissao_usuario_organizacao_id_fkey
FOREIGN KEY (organizacao_id) REFERENCES
folha_ponto.organizacao(id)
);

CREATE INDEX idx_permissao_org_usuario ON
folha_ponto.excecao_permissao_usuario USING btree (organizacao_id,
usuario_id);
```

```
-- Permissions
```

```
ALTER TABLE folha_ponto.excecao_permissao_usuario OWNER TO
backend;

GRANT ALL ON TABLE folha_ponto.excecao_permissao_usuario TO
backend;
```

```
-- folha_ponto.relacionamento definition
```

```
-- Drop table
```

```
-- DROP TABLE folha_ponto.relacionamento;
```

```
CREATE TABLE folha_ponto.relacionamento (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    organizacao_id uuid NOT NULL,
    sujeito_id varchar(64) NOT NULL,
    objeto_id varchar(64) NOT NULL,
    tipo_relacionamento varchar(64) NOT NULL,
    inicio_validade timestamp NULL,
    fim_validade timestamp NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,
    atualizado_por varchar(64) NULL,
    atualizado_em timestamp NULL,
```

```

        CONSTRAINT relacionamento_pkey PRIMARY KEY (id),
        CONSTRAINT relacionamento_organizacao_id_fkey FOREIGN KEY
(organizacao_id) REFERENCES folha_ponto.organizacao(id)
);

CREATE INDEX idx_relacionamento_org_objeto ON
folha_ponto.relacionamento USING btree (organizacao_id,
objeto_id);

CREATE INDEX idx_relacionamento_org_sujeito ON
folha_ponto.relacionamento USING btree (organizacao_id,
sujeito_id);

-- Permissions

ALTER TABLE folha_ponto.relacionamento OWNER TO backend;
GRANT ALL ON TABLE folha_ponto.relacionamento TO backend;

-- folha_ponto.unidade definition

-- Drop table

-- DROP TABLE folha_ponto.unidade;

CREATE TABLE folha_ponto.unidade (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    organizacao_id uuid NOT NULL,
    nome varchar(255) NOT NULL,
    tipo_geometria varchar(16) NOT NULL,
    centro_geo_lat float8 NULL,
    centro_geo_lng float8 NULL,
    raio_geo_metros int4 NULL,
    poligono_geo jsonb NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,

```

```

    atualizado_por varchar(64) NULL,
    atualizado_em timestamp NULL,
    CONSTRAINT unidade_pkey PRIMARY KEY (id),
    CONSTRAINT unidade_organizacao_id_fkey FOREIGN KEY
(organizacao_id) REFERENCES folha_ponto.organizacao(id)
);
CREATE INDEX idx_unidade_org ON folha_ponto.unidade USING btree
(organizacao_id);

```

```
-- Permissions
```

```

ALTER TABLE folha_ponto.unidade OWNER TO backend;
GRANT ALL ON TABLE folha_ponto.unidade TO backend;

```

```
-- folha_ponto.vinculo_usuario definition
```

```
-- Drop table
```

```
-- DROP TABLE folha_ponto.vinculo_usuario;
```

```

CREATE TABLE folha_ponto.vinculo_usuario (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    organizacao_id uuid NOT NULL,
    usuario_id varchar(64) NOT NULL,
    papel varchar(32) NOT NULL,
    data_inicio timestamp DEFAULT CURRENT_TIMESTAMP NOT NULL,
    data_fim timestamp NULL,
    status varchar(32) DEFAULT 'ATIVO'::character varying NOT
NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,
    atualizado_por varchar(64) NULL,

```

```
    atualizado_em timestamp NULL,  
    CONSTRAINT vinculo_usuario_pkey PRIMARY KEY (id),  
    CONSTRAINT vinculo_usuario_organizacao_id_fkey FOREIGN KEY  
(organizacao_id) REFERENCES folha_ponto.organizacao(id)  
);
```

```
CREATE UNIQUE INDEX uq_vinculo_usuario_org ON  
folha_ponto.vinculo_usuario USING btree (usuario_id,  
organizacao_id);
```

```
-- Permissions
```

```
ALTER TABLE folha_ponto.vinculo_usuario OWNER TO backend;  
GRANT ALL ON TABLE folha_ponto.vinculo_usuario TO backend;
```

```
-- folha_ponto.alocacao_unidade definition
```

```
-- Drop table
```

```
-- DROP TABLE folha_ponto.alocacao_unidade;
```

```
CREATE TABLE folha_ponto.alocacao_unidade (  
    id uuid NOT NULL,  
    vinculo_usuario_id uuid NOT NULL,  
    unidade_id uuid NOT NULL,  
    papel_operacional varchar(32) NOT NULL,  
    status varchar(32) DEFAULT 'ATIVO'::character varying NOT  
NULL,  
    data_inicio timestamp NOT NULL,  
    data_fim timestamp NULL,  
    criado_por varchar(64) NOT NULL,  
    criado_em timestamp NOT NULL,  
    atualizado_por varchar(64) NULL,
```

```

    atualizado_em timestamp NULL,
    CONSTRAINT alocacao_unidade_pkey PRIMARY KEY (id),
    CONSTRAINT chk_alocacao_papel CHECK
(((papel_operacional)::text = ANY
((ARRAY['PARTICIPANTE'::character varying, 'MANAGER'::character
varying, 'GESTOR_TITULAR'::character varying,
'GESTOR_SUBSTITUTO'::character varying, 'ASSISTENTE'::character
varying, 'OWNER'::character varying]))::text[]))),
    CONSTRAINT chk_alocacao_status CHECK (((status)::text = ANY
((ARRAY['ATIVO'::character varying, 'SUSPENSO'::character varying,
'ENCERRADO'::character varying]))::text[]))),
    CONSTRAINT fk_alocacao_unidade FOREIGN KEY (unidade_id)
REFERENCES folha_ponto.unidade(id) ON DELETE CASCADE,
    CONSTRAINT fk_alocacao_vinculo FOREIGN KEY
(vinculo_usuario_id) REFERENCES folha_ponto.vinculo_usuario(id) ON
DELETE CASCADE
);
CREATE INDEX idx_alocacao_vinculo ON folha_ponto.alocacao_unidade
USING btree (vinculo_usuario_id, status);

```

```
-- Permissions
```

```

ALTER TABLE folha_ponto.alocacao_unidade OWNER TO backend;
GRANT ALL ON TABLE folha_ponto.alocacao_unidade TO backend;

```

```
-- folha_ponto.convite definition
```

```
-- Drop table
```

```
-- DROP TABLE folha_ponto.convite;
```

```

CREATE TABLE folha_ponto.convite (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    organizacao_id uuid NOT NULL,
    unidade_id uuid NULL,

```

```

    papel_esperado varchar(32) DEFAULT 'PARTICIPANTE'::character
varying NOT NULL,
    codigo varchar(32) NOT NULL,
    data_expiracao timestamp NOT NULL,
    usado bool DEFAULT false NOT NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,
    atualizado_por varchar(64) NULL,
    atualizado_em timestamp NULL,
    CONSTRAINT chk_convite_papel CHECK (((papel_esperado)::text =
ANY ((ARRAY['PARTICIPANTE'::character varying,
'MANAGER'::character varying, 'GESTOR_TITULAR'::character varying,
'GESTOR_SUBSTITUTO'::character varying, 'ASSISTENTE'::character
varying, 'OWNER'::character varying])::text[]))),
    CONSTRAINT convite_pkey PRIMARY KEY (id),
    CONSTRAINT convite_organizacao_id_fkey FOREIGN KEY
(organizacao_id) REFERENCES folha_ponto.organizacao(id),
    CONSTRAINT fk_convite_unidade FOREIGN KEY (unidade_id)
REFERENCES folha_ponto.unidade(id) ON DELETE CASCADE
);
CREATE UNIQUE INDEX uq_convite_codigo ON folha_ponto.convite USING
btree (codigo);

-- Permissions

ALTER TABLE folha_ponto.convite OWNER TO backend;
GRANT ALL ON TABLE folha_ponto.convite TO backend;

-- folha_ponto.registro_presenca definition

-- Drop table

-- DROP TABLE folha_ponto.registro_presenca;

```

```

CREATE TABLE folha_ponto.registro_presenca (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    organizacao_id uuid NOT NULL,
    unidade_id uuid NOT NULL,
    usuario_id varchar(64) NOT NULL,
    latitude float8 NULL,
    longitude float8 NULL,
    precisao_gps float8 NULL,
    referencia_biometrica uuid NULL,
    pontuacao_biometrica float8 NULL,
    biometria_valida bool NULL,
    dispositivo_id varchar(128) NOT NULL,
    modo_registro varchar(16) NOT NULL,
    usuario_intermediario_id varchar(64) NULL,
    pontuacao_risco float8 NULL,
    indicadores_risco jsonb NULL,
    status_tecnico varchar(32) DEFAULT 'RECEBIDO'::character
varying NOT NULL,
    status_administrativo varchar(32) DEFAULT
'PENDENTE'::character varying NOT NULL,
    capturado_em timestamp NOT NULL,
    recebido_no_servidor_em timestamp NOT NULL,
    criado_por varchar(64) NOT NULL,
    criado_em timestamp NOT NULL,
    CONSTRAINT registro_presenca_pkey PRIMARY KEY (id),
    CONSTRAINT registro_presenca_organizacao_id_fkey FOREIGN KEY
(organizacao_id) REFERENCES folha_ponto.organizacao(id),
    CONSTRAINT registro_presenca_unidade_id_fkey FOREIGN KEY
(unidade_id) REFERENCES folha_ponto.unidade(id)
);

CREATE INDEX idx_presenca_org_tempo ON
folha_ponto.registro_presenca USING btree (organizacao_id,
recebido_no_servidor_em);

```

```
CREATE INDEX idx_presenca_org_usuario ON
folha_ponto.registro_presenca USING btree (organizacao_id,
usuario_id);

CREATE INDEX idx_presenca_unidade_tempo ON
folha_ponto.registro_presenca USING btree (unidade_id,
recebido_no_servidor_em);

-- Permissions

ALTER TABLE folha_ponto.registro_presenca OWNER TO backend;
GRANT ALL ON TABLE folha_ponto.registro_presenca TO backend;

-- folha_ponto.alocacao_unidade_aud definition

-- Drop table

-- DROP TABLE folha_ponto.alocacao_unidade_aud;

CREATE TABLE folha_ponto.alocacao_unidade_aud (
    id uuid NOT NULL,
    rev int8 NOT NULL,
    revtype int2 NULL,
    vinculo_usuario_id uuid NULL,
    unidade_id uuid NULL,
    papel_operacional varchar(32) NULL,
    status varchar(32) NULL,
    data_inicio timestamp NULL,
    data_fim timestamp NULL,
    criado_por varchar(64) NULL,
    criado_em timestamp NULL,
    atualizado_por varchar(64) NULL,
    atualizado_em timestamp NULL,
```



```
    rev_end int8 NULL,  
    revend_tstamp timestamp NULL,  
    CONSTRAINT alocacao_unidade_aud_pkey PRIMARY KEY (id, rev)  
);
```

```
-- Permissions
```

```
ALTER TABLE folha_ponto.alocacao_unidade_aud OWNER TO backend;  
GRANT ALL ON TABLE folha_ponto.alocacao_unidade_aud TO backend;
```

```
-- folha_ponto.vinculo_usuario_aud definition
```

```
-- Drop table
```

```
-- DROP TABLE folha_ponto.vinculo_usuario_aud;
```

```
CREATE TABLE folha_ponto.vinculo_usuario_aud (  
    id uuid NOT NULL,  
    rev int8 NOT NULL,  
    revtype int2 NULL,  
    organizacao_id uuid NULL,  
    usuario_id varchar(64) NULL,  
    papel varchar(32) NULL,  
    status varchar(32) NULL,  
    data_inicio timestamp NULL,  
    data_fim timestamp NULL,  
    criado_por varchar(64) NULL,  
    criado_em timestamp NULL,  
    atualizado_por varchar(64) NULL,  
    atualizado_em timestamp NULL,  
    rev_end int8 NULL,
```

```
    revend_tstamp timestamp NULL,  
    CONSTRAINT vinculo_usuario_aud_pkey PRIMARY KEY (id, rev)  
);
```

```
-- Permissions
```

```
ALTER TABLE folha_ponto.vinculo_usuario_aud OWNER TO backend;  
GRANT ALL ON TABLE folha_ponto.vinculo_usuario_aud TO backend;
```

```
-- folha_ponto.alocacao_unidade_aud foreign keys
```

```
ALTER TABLE folha_ponto.alocacao_unidade_aud ADD CONSTRAINT  
fk_alocacao_unidade_aud_rev FOREIGN KEY (rev) REFERENCES  
auditoria.revinfo(id);
```

```
-- folha_ponto.vinculo_usuario_aud foreign keys
```

```
ALTER TABLE folha_ponto.vinculo_usuario_aud ADD CONSTRAINT  
fk_vinculo_usuario_aud_rev FOREIGN KEY (rev) REFERENCES  
auditoria.revinfo(id);
```

```
-- Permissions
```

```
GRANT ALL ON SCHEMA folha_ponto TO backend;
```

5.3 Resumo Técnico e Diretrizes do Schema

- **Isolamento de Locatário (Tenant):** Tabelas cruciais como `organizacao`, `unidade`, `vinculo_usuario`, `convite` e `registro_presenca` carregam consigo índices compostos e amarrações com a tabela raiz de organizações, garantindo que queries possam ser particionadas ou filtradas isolando cada cliente Enterprise.

- **Registro de Eventos Imutáveis:** A tabela `registro_presenca` concretiza a tese do Comando Único. Ela prevê colunas físicas tanto para o momento real do clique no celular (`capturado_em`), quanto o *Timestamp Seguro* blindado do próprio servidor (`recebido_no_servidor_em`). Ela armazena pontuações cruas do motor antifraude (`pontuacao_risco`, `pontuacao_biometrica`), classificando o dado como um fato inalterável.
- **Autorização Dinâmica (ABAC/PBAC):** Este schema detém tabelas vitais para a política de Zero Trust e autorização independente do Keycloak, abrigando as tabelas `excecao_permissao_usuario` e `relacionamento`. Elas servem de espinha dorsal para um Motor de Políticas Dinâmico (Policy Decision Point customizado), registrando quem pode fazer o quê através de colunas robustas (`condicao_json` e `tipo_relacionamento`).
- **Auditoria de Histórico (Padrão Envers):** Enquanto o schema `pessoa` usa triggers de manipulação direta, o schema `folha_ponto` implementa o rastreamento integral (snapshots temporais) das tabelas mais sensíveis de permissões (`alocacao_unidade_aud` e `vinculo_usuario_aud`), atrelando cada modificação a uma tabela de revisão (`revinfo`). Isso cria uma "máquina do tempo" perfeita para saber quais eram as permissões de um gestor há exatos 6 meses atrás.
- **Controle de Consentimento e Dispositivo:** Com a LGPD em mente, tabelas como `consentimento` foram colocadas explicitamente para atestar juridicamente a permissão do tratamento de informações biométricas do usuário em prol da organização alvo. A tabela `dispositivo` orquestra a vinculação e a confiança entre o equipamento de uso e a validação do token (mitigação de hardware).